GUÍA OFICIAL DE TALLER

# Middlewares en Express con TypeScript

Manual técnico para la gestión de solicitudes, blindaje de APIs y división de salas en Zoom.

> 💡 **La Analogía Cotidiana:** Imagina que estás ingresando a un aeropuerto. La ruta final (controlador de datos) es el avión al que quieres subir. El **Middleware** es el oficial de seguridad en la puerta: revisa tu boleto (autenticación), inspecciona tu equipaje (validación de datos) y anota tu hora de entrada (logging) antes de permitirte pasar con `next()`.

## 1. ¿Qué es técnicamente un Middleware?

Una función que intercepta el ciclo de vida HTTP. Tiene acceso a `req`, `res` y a la función `next`. Si todo es correcto, invoca `next()`; si hay fallas, corta la petición respondiendo con un código de error (ej. 401 o 400).

> ⚡ **Tipos en Express:** Aplicación (globales con `app.use`), de Ruta (específicos en endpoints), de Terceros (ej. `express.json()`) y de Manejo de Errores (4 parámetros: `err, req, res, next`).

## 2. Resumen de Endpoints Base (Array simulado)

| Método | Ruta | Descripción Funcional |
|---|---|---|
| GET | /api/productos | Obtiene todos los registros activos. |
| GET | /api/productos/:id | Busca un registro por su ID numérico. |
| POST | /api/productos | Crea un nuevo producto en el array. |
| PUT | /api/productos/:id | Actualiza un producto existente. |
| PATCH/DEL | /api/productos/soft/:id | Soft Delete (cambia `activo: false`). |
| DELETE | /api/productos/hard/:id | Hard Delete (remueve del array). |

## 3. Breakout Room — Sala 1: Validación de Datos (POST / PUT)

**Objetivo:** Blindar la entrada de datos en los endpoints de creación y actualización para evitar que el array reciba registros incompletos o malformados.

### 📋 Tareas de la Sala 1:

- Definir la interfaz `Producto` con `id`, `nombre`, `precio`, `stock` y `activo`.
- Crear el middleware `validarProducto` para verificar tipos de datos en el `req.body`.
- Retornar `400 Bad Request` si falta el nombre o el precio es menor o igual a 0.

### Código Base Guía — Sala 1

```
import { Request, Response, NextFunction } from 'express'; export interface Producto { id: number; nombre: string; precio: number; stock: number; activo: boolean; } export const validarProducto = (req: Request, res: Response, next: NextFunction): void => { const { nombre, precio, stock } = req.body; if (!nombre || typeof nombre !== 'string' || nombre.trim() === '') { res.status(400).json({ error: 'El nombre es obligatorio y debe ser texto.' }); return; } if (typeof precio !== 'number' || precio <= 0) { res.status(400).json({ error: 'El precio debe ser un número mayor a 0.' }); return; } next(); };
```

💡 **Tip de Instructor:** Pide a los alumnos de esta sala que prueben su middleware pasando un JSON vacío en Postman y verifiquen que la respuesta sea el código `400` exacto.

## 4. Breakout Room — Sala 2: Control de Roles (Soft & Hard Delete)

**Objetivo:** Proteger operaciones destructivas en la API diferenciando permisos mediante cabeceras HTTP personalizadas (`x-rol`).

### 🔒 Tareas de la Sala 2:

- Implementar `verificarRol` que lea el header `x-rol`.
- Asignar el borrado lógico (Soft Delete) a roles **MODERADOR** o **ADMIN**.
- Asignar el borrado definitivo (Hard Delete) exclusivamente a **ADMIN** (código 403 si no cumple).

### Código Base Guía — Sala 2

```
import { Request, Response, NextFunction } from 'express'; export const verificarRol = (rolMinimo: 'MODERADOR' | 'ADMIN') => { return (req: Request, res: Response, next: NextFunction): void => { const rol = req.headers['x-rol']; if (!rol || (rolMinimo === 'ADMIN' && rol !== 'ADMIN')) { res.status(403).json({ error: 'Acceso denegado. Se requiere rol ADMIN.' }); return; } next(); }; };
```

🔄 **Plenaria Final (15 min):** Un representante de cada sala unirá su código en el servidor principal con el middleware global de errores `(err, req, res, next)`.